

SOFTWARE QUALITY ASSURANCE

Principles and Practice

Nina S Godbole

Scanned and Compiled By:

Muhammad Shazib

Sufian Ahmad

BS-Information Technology

Batch 2018-2022

Professor: Mr. Muhammad Younas

University of the Punjab.

Software Quality Assurance

Principles and Practice

Nina S Godbole

29 SEP 2012

Preface

Software Quality Assurance (SQA) as a professional domain is becoming increasingly important. A few years back they said, "SQA has a future"; now with a tight competition, we say—"There is no future for business without Software Quality Assurance practices built in". This book fulfills the long awaited need of the Information Technology/Software industry for a book on SOFTWARE QUALITY ASSURANCE.

There is no dearth of books on Software Engineering, which also cover the topic of Software Quality Assurance. But this book is entirely devoted to the topic of *Software Quality Assurance*. It starts with a discussion on importance of software quality assurance in the business of Information Technology. It covers key practices like Verification and Validation. It also discusses people issues and other barriers in successful implementation of Quality Management Systems in organization.

The basic premise here is that quality can be achieved through continuous process improvement and that product quality is possible through high quality of the processes used to produce the product. Thus, Quality Assurance, in its intent, is different than Quality Control (QC). While QC is an operational issue, Quality Assurance (QA) is a strategic issue requiring due attention from top management. QA is a proactive approach to ensure quality by establishing meaningful and adequate processes, to be followed throughout the life cycle. There are several definitions for the simple word "quality" - however, in reality it is not so simple when it comes to implementing it. My working experience as a quality professional (as well as a systems analyst and developer in earlier years) provided me opportunities to have a closer look at the quality scenario. This book is an attempt to explain the underlying quality assurance concepts in a way that would appeal to large diversity of readers.

A quality professional may successfully lead several cycles of verification and various types of quality audits and assessments. For me, too, my job responsibility provided an enriched experience through hands-on work in ISO audits, CMM Assessments, People CMM and also in the arena of the Integrated CMM. However, through and through, amidst these various implementations as part of organizational directives and initiative, there was a growing conviction that quality is a

"state of mind" a matter of "culture" and deep "institutionalization". As such, it needs to become a "way of life".

It looks like in its true sense; quality culture is yet to come in the software industry. The sheer change of designation from a "Programmer" to a "Software Engineer" or the mere shift in role responsibility from "coder" to "software design architect" may not bring in the right kind of mindset and mature attitude to quality in software development and service organizations. It is a thankless task that a true quality professional needs to take upon himself/herself to influence many non-believers of quality into strong believers in quality; the challenge lies in achieving this without going into conflicts or without creating abrasive situations.

Quality Management is an integral part of Project Management. However, Project Managers often fail to recognize/understand this. In the pursuit of acquiring quality certifications to flaunt them at the customers, one must not lose sight of quality. Quality Assurance has yet not earned its due place in software industry. There are reasons to believe that most Indian organizations go for quality certifications due to marketing/business pressures, and not much due to their firm belief in the concept of continuous process improvement as an on-going quest. It must be appreciated that Quality makes a business case and as such, it should be part of the strategic business plan.

The other motivation behind this work is the finding that there are not many titles on this subject by practicing quality professionals. Thus, there was a need for an Indian author to take this initiative towards writing a book on SQA that would serve a two-fold purpose; on one hand, it would serve as a rich source of SQA philosophy and discipline to champion the quality cause among the young students graduating out of engineering colleges and management schools, and on the other hand, a book rich with a practitioner's experience. It's with this strong conviction that this book-writing endeavor was taken up. It took two years to complete this work—whereas for a full-time writer perhaps, it'd have taken much less. Writing a book while holding a busy working schedule is not easy! Priority to the job often result in long hours at work.

Motivation for writing this book also came from the experience of imparting SQA skills to the aspirants of quality certification who joined the study circles devoted to this purpose. During this association, interactions with the working professionals from industry, threw open several eye-opening, mind-boggling questions. This provided an impetus to constant thinking and deep introspection.

The book is organized into 11 chapters and 3 appendices. It can be read cover-to-cover as well as in combinations of certain chapters. The first three chapters form a theme together to provide a *business context to Quality*. Chapter 4 and 5 should be read together as the topic of *measurements and metrics* is related to the notions of *product quality* and

process quality. Reading chapter 8 and 9 together would form another cohesive theme on *process improvement*. Chapter 10 on *testing* is to satisfy the needs of those who are looking for a focus on *quality control*. Chapter 6 on *Walkthroughs and Inspection* along with the chapter on testing would form another coherent theme as these review techniques are considered "static testing". Chapter 11 helps paint a canvas in the light of the rest of discussion in the book, on career decisions. The three appendices are stand-alone references for providing allied information related to the topic of Software Quality Assurance.

One of the aims for this book is to spread the message that embedded quality practices can make a difference between lost revenues and improved profitability, between delayed release and speedier delivery, between poor product performance and true customer satisfaction. This book will have served its purpose if it motivates practitioners towards a process-oriented approach to establish Software Quality Assurance in their organization and if it can help create a conviction about the subject in the mind of quality professionals and finally if it creates many "quality champions" out of education campuses.

Nina Godbole

Acknowledgements

Book writing is a daunting task; it is a project and as such requires collaboration from a multitude of people apart from the author's own will and skill! At times, it may even seem that the efforts will never go through and that's when one needs a few encouraging words of support to keep up one's enthusiasm level.

The following individuals mattered for completing this book-writing project. All of them did their bit to help—either directly or indirectly: in one form or the other. Some provided valuable review comments; others offered to help or keep me company during the long hours of arduous proof reading. Some others helped co-ordinate meetings with various individuals who needed to be consulted during the two-year tenure of this endeavour. Some were a remote source of inspiration and role models as authors, while some just kept me going with their benign smiles. I was fortunate also to find the guidance and time of some very busy individuals, well known in the field of Information Technology. I take this opportunity to express my gratitude to all of them for their valuable support.

N.K. Mehra
Prabha Ananthanarayanan
Karunanidhi O. Aiyar
Gautam Modi
Ketaki Godbole
Dr. Sundeep Oberoi
Nitin Yeravadekar
Dr. Bhuvan Unhelkar
Dr. Shashikant Kelkar
Sagar Joshi
Dr. Asha Pachpande

Dr. D.B. Phatak
Narendra Barhate
Dr. Asha Goyal
Uday Panchpor
Sudhir Godbole
Gireendra Kasmalkar
Alok Mohgaonkar
Dr. Athula Ginige
Pramod Paranjpe
Vishwas Mahajan
Anupama Jauhry

CONTENTS

<i>Foreword</i>	v
<i>Preface</i>	vii
<i>Acknowledgements</i>	xi
Chapter 1: Software Quality in Business Context	1
1.1 The Meaning of "Quality": Defining Quality	1
1.2 The Quality Challenge	2
1.3 Why is Quality Important?	3
1.4 Quality Control v/s Quality Assurance	7
1.5 Quality Assurance at each Phase of SDLC	8
1.6 Quality Assurance in Software Support Projects	18
1.7 The SQA Function	19
Chapter 2: Managing Software Quality in An Organization	25
2.1 Quality Management System in An Organization	25
2.2 Quality Management System: Various Expectations	31
2.3 Quality Assurance: Some Diagnostic Questions	38
2.4 The Need for the SQA Group in An Organization	46
Chapter 3: Planning for Software Quality Assurance	55
3.1 Software Quality Assurance Plans	55
3.2 Software Quality Assurance: Organizational Level Initiatives	67
3.3 Quality Planning—Some Interesting Dilemmas and Observations	76
Chapter 4: Product Quality and Process Quality	80
4.1 Introduction	80
4.2 Software Systems Evolution	81
4.3 Product Quality	82
4.4 Models for Software Product Quality	84
4.5 Process Quality	88

Chapter 5:	Software Measurement and Metrics	96
5.1	Overview	96
5.2	Introduction	97
5.3	Measurement during Software Life Cycle Context	102
5.4	Defect Metrics	106
5.5	Metrics for Software Maintenance	110
5.6	Classification of Software Metrics	114
5.7	Requirements Related Metrics	115
5.8	Measurements and Process Improvement	119
5.9	Measurement Principles	120
5.10	Identifying Appropriate Measures and Metrics for Projects	125
5.11	Metrics Implementation in Projects	132
5.12	Benefits of Measurement and Metrics for Project Tracking and Control	135
5.13	Earned Value Analysis	137
5.14	Planning for Metrics Program	141
5.15	Issues in Software Measurements & Metrics Program Implementation	142
5.16	Object Oriented Metrics: An Overview	146
Chapter 6:	Walkthroughs and Inspections	155
6.1	Overview	155
6.2	Introduction	155
6.3	Structured Walkthroughs	156
6.4	Inspections	158
6.5	Various Roles and Responsibilities involved in Reviews/Inspections	159
6.6	Some Psychological Aspects of Reviews	162
6.7	Making Reviews and Inspections Effective	165
6.8	Comparison of Review Techniques	167
6.9	Inspection Related Checklists	169
Chapter 7:	Software Configuration Management	173
7.1	Overview	173
7.2	Configuration Management: Why and What	174
7.3	Software Configuration Management Activities	177
7.4	Standards for Configuration Audit Functions	192
7.5	Personnel in SCM Activities	197
7.6	Software Configuration Management: Some Pitfalls	197

Chapter 8: ISO 9001	200
8.1 Overview	200
8.2 What is ISO 9000	200
8.3 The Origins of ISO 9000	201
8.4 How Does ISO (as an organization) carry out its work?	202
8.5 ISO Standards Development Process	203
8.6 How the ISO 9000 Family of Standards Work	205
8.7 ISO 9001:2000	205
8.8 Why do Organizations Need ISO 9000?	207
8.9 ISO Certification	208
8.10 Assessment/Audit Preparation	212
8.11 The Assessment Process	213
8.12 Surveillance Audits/Re-Certification/Re-Assessment Audits	215
8.13 ISO Consulting Services and Consultants	215
8.14 ISO 9000: Some FAQs	221
8.15 e-Business and ISO	222
Chapter 9: Software CMM and Other Process Improvement Models	224
9.1 Overview	224
9.2 The Capability Maturity Model for Software: An Overview	224
9.3 Practices Followed at "Mature Organizations"	227
9.4 CMM and ISO (A Comparative Analysis)	227
9.5 Types of Capability Maturity Models (CMMs)	234
9.6 The CMM-Integrated Model (CMM-I)	235
9.7 Other Models for Software Process Improvement and Performance Excellence	244
9.8 The People Maturity Model (P-CMM)	252
Chapter 10: Software Testing	257
10.1 Overview	257
10.2 Purpose of Testing	258
10.3 Differences Between Inspection and Testing	260
10.4 Testing v/s Debugging	262
10.5 Testing Life Cycle	264
10.6 Roles and Responsibilities in Testing	266
10.7 Test Artifacts	269
10.8 The Test Plan	273
10.9 The V-Model for Testing Phases	275
10.10 Testing Techniques	279
10.11 Test Metrics	289
10.12 Risk-Based Testing	293
10.13 Test Automation and Test Tool Selection	297
10.14 Extreme Testing	300

10.15 Test Process Improvement Framework	301
10.16 Human Issues and Challenges in Testing	306
10.17 Software Testing Careers and Testing Competence Through Professional Certifications	308
Chapter 11: Careers in Quality	311
11.1 Overview	311
11.2 Introduction	311
11.3 P-CMM and Careers	312
11.4 Some important "People Issues"	312
11.5 Finding a Mentor to Shape Your Career	313
11.6 Roles for Quality Professionals	314
11.7 Quality Certifications	320
Appendix A: Process Improvement Related Miscellaneous Topics	325
A.1 Some Basic Terms	325
A.2 High Maturity Organizations (those at SW CMM Level 4 or 5)	328
A.3 Strategic Planning for Software Process Improvement	334
A.4 Software CMM for Small Organizations	335
A.5 Mini Assessment Process	337
A.6 The Integrated Capability Maturity Model (CMM-I)	338
Appendix B: Indian Software Industry in Perspective	341
B.1 Quality Certifications Scenario in India	341
B.2 The Indian Software Scenario in General	350
B.3 Cost and Quality Advantage: Indian Software Industry	351
B.4 The Indian Software Industry: Problems & Challenges	352
Appendix C: Quality Related Additional Topics	360
C.1 Statistical Quality Control and Statistical Process Control	360
C.2 Software Maintenance Models: The Evolution	366
C.3 OO Maintainability	371
C.4 Maintenance in e-Business Era	372
C.5 Cyclomatic Complexity	383
C.6 Requirements	386
C.7 Principle of COUPLING and COHESION	388
C.8 Six Sigma	402
Index	406

Chapter 1

SOFTWARE QUALITY IN BUSINESS CONTEXT

1.1 The Meaning of "Quality": Defining Quality

The term "Quality" is one of the most discussed terms but is often not defined crisply. All practitioners agree that "Quality" is a major business factor, a distinguisher that makes the difference and yet the term is extraordinarily difficult to define precisely. Quality is easy to see and is immediately apparent when encountered or missing! But when you try to pin it down or define what it is, you find that the concept is elusive. It is not the complexity, but the utter simplicity of quality that defies its explanation.

Here are several excerpts from views on this matter from various software quality specialists whom we refer to as the "Quality Gurus":

- Dr. Barry Boehm thinks of quality as "Achieving high levels of user satisfaction, portability, maintainability, robustness, and fitness for use"
- Phil Crosby has created the definition with the widest currency because of its publication in his famous book "Quality is Free". He states that quality means "conformance to user requirements"
- Edwards Deming considers quality to be "striving for excellence" in reliability and functions by continuous improvement in the process of development, supported by statistical analysis of the causes of failure."
- Watts Humphrey, of the Software Engineering Institute (SEI), tends to speak of quality as "achieving excellent levels of fitness for use, conformance to requirements, reliability and maintainability."
- James Martin has asserted that software quality means being on time, within budget and meeting user needs.
- Tom McCabe, the software complexity specialist, defines quality as "high levels of user satisfaction and low defect levels, often associated with low complexity"

- John Musa of Bell Laboratories states that quality means combination of "low defect levels, adherence of software functions to users needs, and high reliability."
- Bill Perry, head of Quality Assurance Institute has defined quality as "high levels of user satisfaction and adherence to requirements".

Each of these definitions, individually, has its own merits and presents a view point on quality. Taken together, all these definitions resembles the "the blind men and the elephant": each is a true aspect of quality. An end user's definition of quality would be "absence of defects that would make software either stop completely or produce unacceptable results". Defects can be traced to any stage in the Software Development Life Cycle (SDLC): requirements, design, code, documentation or to bad fixes of previous defects. Defects can range in severity levels. Thus, a working definition of quality must meet two important criteria:

- Quality must be measurable when it occurs
- Quality should be predictable when it occurs

Table 1 below lists the elements of quality definitions quoted above and indicates which elements meet both criteria.

Table 1: Elements of Quality Definition

Quality Factor	Predictable	Measurable
Defect Level	Yes	Yes
Defect Origins	Yes	Yes
Defect Severity	Yes	Yes
Defect Removal Efficiency	Yes	Yes
Product Complexity	Yes	Yes
Project reliability	Yes	Yes
Project maintainability	Yes	Yes
Project schedules	Yes	Yes
Project budgets	Yes	Yes
Portability	Yes	Yes
Conformance to Requirements	No	Yes
User Satisfaction	No	Yes
Fitness for Use	No	Yes
Robustness	No	No

1.2 The Quality Challenge

The goal of software quality is elusive in an actual project environment. The measures differ from project to project and organization to organization. Criteria vary as a function of the specific characteristics of the project, the needs of the users and stakeholders, and the application requirements of the system and software. Quality measures used for

small systems may not be appropriate for the large ones. Criteria for quality applied to real-time applications are not always relevant when dealing with systems that are not real-time. Complex software requires different monitoring procedures than trivial applications. Quality criteria vary dramatically depending on the phase of the project at which the evaluation takes place (later in this chapter, quality assurance at each SDLC stage is discussed). The measures of quality must be specific to the project being evaluated and must assess the effectiveness of the entire development process, not just individual segments.

It is said that quality cannot be directly checked in the product; it must be planned right from the beginning. Thus, software quality must be planned into the project right from the initiation stage, engineered into the products of the software development project, and be monitored by assessing not only individual segments of the project or single data products but also by evaluating the interactions and interrelationships between them. Quality goals must be clearly defined, effectively monitored, and rigorously enforced. The project must focus on the quality issues of the project from the outset, ensuring that quality criteria are consistent with defined requirements. Throughout software development, the management of quality must be an overriding concern of all project personnel. Quality must be planned into the project structure, constantly evaluated, and corrections applied when deficiencies are identified.

1.3 Why is Quality Important?

Why do we need to bother with quality? Because today, quality is critical for survival and success. Customers demand it. Software is now a global business and organizations will not succeed in this global market unless they produce quality products/services and unless the organizations are also perceived by customer to produce quality products and services. This is the first message of this chapter. It applies as much to software development and support as to any other product or service (ISO definition of product includes services as well). However, there is something very interesting when it comes to a discussion on the topic of "quality": everybody seems to understand it, everybody wants it and yet everyone has a different perception of quality. Earlier in this chapter we had provided various definitions of quality, as originated from various quality gurus; essentially quality means satisfying customer. A happy customer will get you repeat business and business houses agree that the cost of getting a new customer is much higher; so much that businesses would like to focus on getting more and more business from the same customers. The entire CRM world (Customer Relationship Management) harps on this aspect. Thus, there are several reasons why businesses should be concerned with quality:

- Quality is a competitive issue now
- Quality is a must for survival
- Quality gives you the global reach
- Quality is cost effective
- Quality helps retain customers and increase profits
- Quality is the hallmark of world-class business

Now let us discuss each of these arguments.

1.3.1 Quality: A Competitive Issue

Several years back, software was considered to be only a technical business, in which functionality was the key determinant of success. Today, you can no longer rely on functionality alone of your products to be a winner. Your competitors can match your functionality relatively quickly and easily. The only way to differentiate your product from those of your competitors, in the long run, is by its quality, and the quality of support that goes with it.

As the software market matures, customers want to be assured of quality. They no longer accept your claims at face value; they expect you to demonstrate quality at each step of their business relationship or interaction with them. Certification to international quality standards is becoming a pre-requisite for getting business.

This does not apply only to the products developed for the external customers; it applies equally well to in-house information systems development. Internal customers also want quality assurance (for example the Help Desk function established in organizations' Information System department goes to prove this point). Increasingly, there are discussions whether the work should be carried out in-house or outsourced to external suppliers. Even while selecting these suppliers, businesses would give a prime consideration to the fact that they are certified in quality or not.

1.3.2 Quality: An Issue for Survival

As said earlier, customers are demanding demonstrable quality. If you cannot deliver it, your ability to survive in a higher competitive and rapidly changing market is in doubt. More and more large organizations are deciding to reduce the number of suppliers they deal with. In the drive to improve their own quality, they want to work closely with their few key suppliers, whom they treat as business partners. They often use quality certification as a way of selecting suppliers.

1.3.3 Quality: For Entry into International Markets

Software is a global business now where players can enter notwithstanding their organization size. The ability to demonstrate quality gives

even a smaller company the credibility to enter an export market. Today, there are many Indian companies in the global market; some of them very small in size in terms of employee numbers but they are dealing with software giants like Microsoft. Although, most of the Indian software houses took quality certification only from the point of entry into international business, today it has become a way of working. Thus, the game of quality works both ways—your home market is vulnerable to quality foreign imports unless you can compete on quality.

1.3.4 Quality is Cost-effective

An effective quality system leads to increased productivity and permanently reduced costs, because it enables management to reduce defect correction costs by emphasizing prevention. Everyone in the software industry knows that the cost of correcting defects in software late in development life cycle can be orders of magnitude greater than the cost of correcting them early. Preventing defects in the first place can save even more. It is possible to achieve this. Leading companies, are aiming at "6 Sigma" quality in the near future.

The scope for reducing costs can be demonstrated by applying the "cost of poor quality" technique. This is a way of analyzing business processes to identify targets for improving initiatives. It is applicable to a software organization as to any other business. Organization's costs can be broken down into the following categories:

- Basic costs inherent in doing the work
- The cost of low quality, comprising:
 - Prevention costs, incurred to prevent defects from occurring, for example training or corrective action aimed at eliminating the causes of failure
 - appraisal costs, incurred for example reviewing work-products, testing software to ensure it meets requirements
 - failure costs, incurred in correcting defects found by appraisals, or by customer after delivery (post delivery defects or customer reported defects)

The cost of low quality can amount to more than 50% of total costs in software organizations. Large savings can be made by analyzing these costs. Large savings are possible by analysis of these costs, and targeting resources to reduce them. Typically, this will be achieved by redirecting resources towards prevention, as illustrated in the figure 1. This applies as much to in-house information systems departments as to commercial software organizations.

The initial costs of establishing a quality system are more than compensated for subsequently, and significant return on investment can be

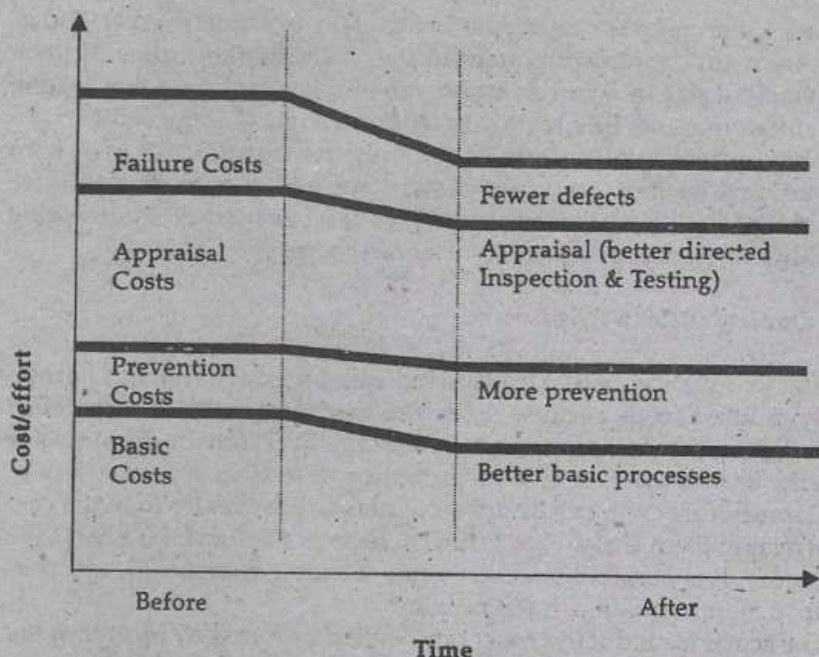


Figure 1: Costs of Poor Quality

expected. For example, there are reports saying that a well known engineering equipment division was able to reduce failure costs over a three year period from about 45% to 15%, and the cost of low quality by 20%, by improving their software engineering process. The savings in the cost of low quality flow straight through to a firm's bottom line, and can dramatically increase profits.

1.3.5 Quality for Retaining Customers and Increasing Profits

Poor quality often costs customers much more than it costs suppliers. Most customers will not tolerate this and will place their business elsewhere. It is said that a silent customer is more dangerous than a complaining customer; you will never know when you lost him! Better quality leads to improved customer satisfaction, delightful customer relationship, and ultimately results in year-on-year retention of existing customers.

As said earlier, the sales cost of attracting a new customer is high. Studies from management consulting firms show that for a software company a 5% increase in customer retention increased profits by 35%.

1.3.6 Quality as the Hallmark of Global Business

World-class businesses place a strategic emphasis on quality. This enables them to grow their business and outperform their competitors.

The reason for this is illustrated in figure 2 below. It shows the "chain reaction" advocated by Deming, one of the quality gurus.

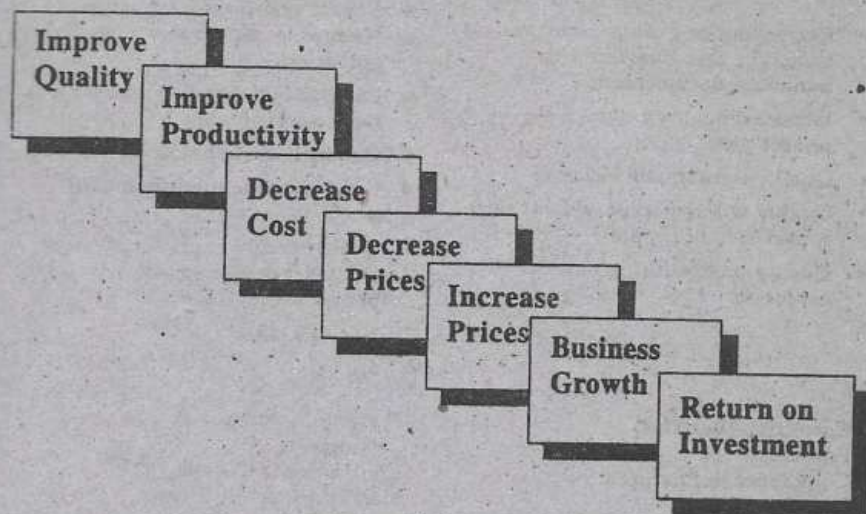


Figure 2: Deming's Chain Reaction

Today, view towards quality has drastically changed. Figure 3 below shows how the scenario stands now.

Changing Views of Quality

Past

- Quality is the responsibility of blue collar workers and direct labour employees working on the product
- Quality defects should be hidden from the customers and management
- Quality problems lead to blame, faulty justification and excuses

Present

- Quality is everyone's responsibility, including white-collar workers, the indirect labour force and the overhead staff
- Defects should be highlighted and brought to the surface for corrective action
- Quality problems lead to co-operative solutions

Figure 3

1.4 Quality Control v/s Quality Assurance

Often, this issue is confused. The two approaches are complementary to each other but different. Whereas QC is corrective, QA is a preventive approach. Quality Control is the operational techniques and activities that are used to fulfill requirements for quality. It includes the processes & methods used to monitor work and observe whether requirements/standards are met. On the other hand, Quality Assurance is all those planned and systematic activities implemented within the quality system and demonstrated as needed to provide adequate confidence that work product will fulfill requirements for quality.

Changing Views of Quality

Past	Present
• Corrections-to-quality problems should be accompanied with minimum documentation • Increased quality will increase project costs • Quality is internally focussed • Quality will not occur without close supervision of people • Quality occurs during project execution	• Documentation is essential for "lessons learnt" so that mistakes are not repeated • Improved quality saves money and increases business • Quality is customer focused • People want to produce quality products • Quality occurs at project initiation and must be planned for within the project
Quality Control v/s Quality Assurance	
QC	QA
Correction (Reactive) Product Confidence to Producer Line Function Find Defects	Prevention (Proactive) Process Confidence to Customer Staff Function Prevent Defects
Examples	
• Walkthrough • Testing • Inspection • Checkpoint review	• Defining Processes • Quality Audit • Selection of Tools • Training

Figure 3

Figure 3 above makes the difference clear.

1.5 Quality Assurance at each Phase of SDLC

Quality during ANALYSIS

There are three major activities that foster quality during the Requirements Analysis phase of a software development project: (1) measurement of process attributes (2) Verification and validation, and (3) management. In fact, these themes will be followed throughout each of the succeeding life cycle stages

We are in a paradoxical situation when we try to measure accuracy, completeness and rationality, the attributes of analysis. We really will not know how well analysis has succeeded until the end of the project. It will be very useful, however, if we compare the predicted schedule, cost, feasibility and market acceptance, staffing needs and so forth with actual figures. This can be done as part of the project debriefing and any variances between planning and actuality can be fed back into the analysis methodologies.

Here, it is important to mention two important terms: VERIFICATION and VALIDATION. Verification refers to the set of activities that ensure that software that software correctly implements a specific function. Validation refers to a different set of activities that ensure that the software that has been built is traceable to customer requirements. In other words:

Verification: "Are we building the product right?"

Validation: "Are we building the right product?"

Verification during analysis can be performed by making sure that functional and non-functional requirements and constraints, as in the requirements specification document are traceable to customer requests that were determined during systems analysis. It is also important to verify risk analysis, cost estimation, strategy and model selection and so on, by comparing them with the results of previous projects. Validation can be performed by allowing users to review the project plan and experiment with prototypes.

Managing quality in the analysis stage is a challenge because requirements may not be very clear at this state. It is at this point that the most important management decisions for the rest of the project will be made. The major management deficiencies in most software development projects seem to be incorrect schedules, incorrect cost estimates, inadequate project accountability procedures, inadequate quality assurance procedures and imprecise goals and success criteria—all products of the ANALYSIS stage

What is a Requirement?

A requirement is a specification of what a system/application must do—the things about the system users can observe. Good requirements have three primary characteristics:

1. They are precise, with no room for misinterpretation by users or implementors.
2. They specify just what the system must do, not how to do it. They avoid specifying implementation details.
3. They show conceptual integrity, building on a simple set of facilities that interact well with each other.

From the user's point of view, the last property is the most important. Conceptual integrity is the key to making a system users can understand. If it has too many features or if each facility has its own peculiarities, the user has an enormous task to understand the system. The key to conceptual integrity is to have one architect or at most a few architects. More than about three people can hardly agree when to meet, let alone what to do!

While defining systems requirements, one may also define constraints and goals for the system. Constraints can also be referred to as "nonfunctional requirements". A constraint is a limitation on possible implementation of the system. For example, a customer may require a particular implementation language, a particular search algorithm say for product search feature of the on-line catalogue module of a web application, or a particular format for a database table etc. in a way not visible to the end users. It is often in our interest to negotiate away constraints since they limit implementation freedom. However, since the customers are paying for the product, they normally succeed in imposing constraints if they really want them.

A goal is a statement that guides tradeoffs among design decisions; For example, a customer may care a lot about maintainability of the software, but not care much about efficiency. A goal may become a requirement if you find a way to quantify it, which in turn allows introduction of measure for improvement. For example, "high throughput" is a goal but "at least 53 transactions per second" is a requirement.

Since requirements are specifications of the observable behaviour of the system, an "implementation detail" is any property of the system that should not be visible to users. Some people use "requirement" to mean any property of the system they care about, and "implementation detail" to mean any property they don't care about. This can lead to confusion. It is better to use taxonomy of this terminology to promote clarity for communication with the users/client during the analysis stage of a software project. Table 2 shows a way of distinguishing among these various terms discussed so far (goal, constraint, requirement).

Using the terminology depicted in above table, an implementation detail the customer cares about is a constraint. A requirement the customer does not care about is simply a requirement that is likely to change. If you list several alternatives, the customer may choose one, or at least reject a few. If the customer still does not care what choice you make, you may discover after delivering the system that the users are unhappy with your choice. Subsequently, during the system design stage, one should try to minimize the consequences of changing this particular requirement.

Quality during DESIGN

Design is pivotal activity in a software development project. A lack of quality in the design process can invalidate an otherwise good requirements specification and can make correct implementation impossible. For this reason, we must take special care to ensure that the design process is carried out correctly.

We'll now discuss the term "metrics" as described in SEI Curriculum Module SEI-CM-12-1.1. (A more detailed discussion will be

Table 2: A Taxonomy of Requirements Terminology

	Customer Doesn't care	Customer Cares	
		Measurable	Un-measurable
Observable to Users	Requirements likely to change	Requirement	Goal
Not observable to Users	Implementation detail	Constraint	

undertaken in the chapter on Software Metrics) As per this published report of December 1988, essentially, SOFTWARE METRICS deals with the measurement of the software PRODUCT and PROCESS by which it is developed. Software metrics may be broadly classified as either PRODUCT METRICS or PROCESS METRICS. Product Metrics include SIZE METRICS (Lines of Code or Function Points) and Complexity Metrics, QUALITY METRICS (Defect Metrics, Reliability Metrics, Maintainability Metrics). Process Metrics, on the other hand, are measures of the development process such as development time, type of methodologies used, or the average level of experience of the programming staff. There are other metrics, too, apart from the ones mention in the SEI literature.

To evaluate design, it is necessary to choose a set of design attributes that will be measured along with some objective metrics, which can be applied in a procedural way to the design. There should also be a procedure for combining individual metrics to evaluate overall design quality. There are several metrics that can be used during the design stage: cyclomatic complexity of a software module (McCabe), the notion of Coupling & Cohesion. One of the fundamental principles of Structured Design is that a large system should be partitioned into manageable modules. However, it is vital that this partitioning should be carried out in such a way that the modules are as "independent" as possible—this is the criterion of COUPLING. Design should ensure that each module carries out a single, problem related function—this is the criterion of COHESION. Readers should make a note that the difficulty of understanding a program is largely determined by the complexity of the control graph for that program.

In an object-oriented design paradigm, the visibility of an object can be taken as a measure of the degree to which it is global to the entire system (Public and Private Access Designations). Generally, a high level of visibility is desirable for more than few objects in the design, and then only those most central to the software (principle of data hiding in object oriented methodology). Industry practice shows that use of checklist during design helps improve design quality. We have provided below one example of such a checklist.

Checklist for Software Design Description

- Have we addressed all the requirements mentioned in the SRS ?
 - Has the SRS been put under document control ?
 - Have the requirements related to the following features been addressed during the design?
 - performance
 - security
 - concurrency
 - usability
 - portability
 - testability
 - language/DB/OS/hardware requirement
 - development environment
 - compatibility (backward/existing products)
 - adherence to industry standards
 - scalability/expandability
 - exception handling
 - Is the design methodology chosen appropriate for the type of software project to be developed?
- a) Clarity
- Is the design documentation clear/unambiguous?
 - Can the design be technically justified?
- b) Compatibility with existing software (if applicable)
- Have the effects of this design on existing software been identified? Have we done the impact analysis?
 - Does this design rely on the side effects of other software? Does this design have any dependency on any other related designs?
- c) Component level
- Are the interfaces well defined? (have we taken care of coupling and cohesion principles?)
 - Are the main data structures defined?
 - Are the main algorithms defined?
 - Is the data/control flow defined?
- d) Data structures and algorithms
- Are the data structures defined?
 - Are the access methods to the data structures defined?
 - Are the algorithms defined?
 - Do the data structures and algorithms solve the problem?
- e) Error/exception handling
- Are data type errors handled?
 - Does the software validate user input?

- Does the software give explicit, non-threatening messages if error occurs? (Quality of error messages)
- Can the software be restarted from any point after an error?
- Does the software gracefully handle exception conditions such as access violations and floating point errors?
- Is the error handling consistent?

f) Procedure interfaces

- Does the number of actual parameters match the number of formal parameters?
- Do the type and size of actual parameters match the type and size of formal parameters?
- Have we specified the local and global functions correctly?
- Are global variables defined and used consistently across modules?
- Is all communication documented (i.e. parameters and shared data)?

g) Procedure level

- Does the procedure do something very similar to an existing procedure?
- Is there a library procedure that will do the same thing?
- Is the procedure excessively complex?
- Could the procedure be broken into separate, more logical pieces?
- Is the procedure of acceptable size?
- Does the procedure do only one logical thing? (Principle of COHESION)
- Does the procedure rely on procedure scope static variables?
- Is the procedure easily maintained and correctly referenced?
- Can the procedure be easily tested?
- Are the side-effects described?

h) Quality

- Are the goals of design stated (reliability, flexibility, maintainability, performance etc.)?
- Does the design satisfy its stated goals? (Traceability to REQUIREMENTS)
- Is there evidence that more than one design option was considered?
- Are several design options listed along with the reason for their adoption or rejection?
- Are the design assumptions stated?
- Are the design trade-offs stated?
- Is the design efficient?
- Is the design maintainable?
- Is the design portable?
- Can the design handle changes to the external environment with minimum modifications?
- Is the design parameter driven or are the values hard coded in the programs?

i) Requirements

- Does the design satisfy every requirement?
- Is there traceability between the design and the system specifications?
- Can the design meet development cost requirements?
- Can the design be completed in the given time frame?
- Does the design stay within the required memory constraint?
- Does the design stay within the required disk usage constraint?
- Does the design satisfy the response time requirement?
- Will the design handle the expected rate of transactions?
- Will the design handle the expected data flow volumes?

j) Reuse

- Does this functionality exist within another procedure or library function?
- Is this functionality sufficiently general purpose that a library function should be created?

k) System level

- Can this design be implemented on the given hardware platform and operating system?
- Can this design be easily ported to other hardware platforms and operating systems?
- Does this platform handle the required terminals, workstations, PCs and printers?
- Does the design allow outputs on a variety of printers?
- Will the design allow for easy installation and configuration?
- Does the design have full international capabilities? Does it handle:
 - Translated text and error messages.
 - Date/time formats and separators.
 - Numeric separators.
 - Sorting sequences.
 - Input/recognition of accented characters etc.

l) User interface

- Had the user interface been prototyped in collaboration with the users?
- Is the software intuitive, consistent, and predictable?
- Does it cater to the various kinds of users—from beginners through experts?
- Can commands be abbreviated?
- Does the program provide sensible defaults to avoid redundant typing?
- Does the software provide context sensitive help?
- Is there detailed help information available on each command?
- Can users cancel time-consuming operations?

- Is the menu hierarchy logical and simple?
- Can the users perform more than one action while in a particular screen or do they have to exit and back up a menu hierarchy?
- Is it easy to back out, wherever the user may be in the program?
- Is it obvious what format to enter information/date in?

Quality during Specification

As the technical foundation of software development, the specification work products should exhibit the highest quality possible. Unfortunately, this is not so. Attention to specifications often gets less focus than other developmental stages. Hence, not much can be said about metrics, verification & validation, and management as they apply specifically to the specification stage in the life cycle of a software project. The irony is that, since all other work products are formally verified against the specifications, it is appropriate to consider here the groundwork necessary for future verification of design and code against specifications.

Quality Indicators for Specification

First, let us explain the difference between the two terms MEASURE and METRICS. MEASURE(ment)—a NUMBER with a DIMENSION—for example: "5 defects" is a MEASURE. METRIC is two related MEASURES—for example: "6 Defects/KLOC is a METRIC. (This is explained in great details in the chapter devoted to Metrics and Measurement)

There are a few measures that are used for indicating the Software Requirements Specification. The first measure is simply whether it conforms to the standard that has been accepted by the software organization. This requires a simple check-off procedure to determine whether each part of the document required by the standard is actually present, whether standards in notation and style are met and so on.

The most obvious attributes for specifications are listed below:

- Correct
- Consistent
- Complete
- Unambiguous
- Minimal
- Formal
- Verifiable
- Transformable
- Modifiable
- Traceable

The quality of the Software Requirements Specification (A good reference for this is : IEEE Guide to Requirements Specification 830) can be evaluated with regard to several different attributes. The level of these attributes in a given specification will determine its appropriateness and usefulness for the task. Let us now discuss each of the above attributes.

The first, and by far the most important attribute is correctness of requirements important to the customer and that are stated in the Software Requirements Specification (SRS) The SRS should have backward traceability to the Contract/Document of Understanding and the SoW—Statement of Work) must appear in correct form in the Software Requirements Specification. Unfortunately, since the customer's requirements were probably stated in an informal language or because may be he was not articulate enough, there are no formal method for verifying the correctness of the SRS (Refer Table 1). This specification can, however be validated by allowing the user to exercise the means of a prototype.

Consistent, Complete and Unambiguous are the attributes that can be logically grouped together. Consistency means we are sure that requirements which are correct in isolation can still be achieved when taken together. Completeness guarantees that no important requirement specification has been left out. Use of checklist helps in this. Unambiguous means that everyone on the project (all stakeholders included) will agree on the exact meaning of the specification..

Minimal means we must guard against over specifying software at this point of time. This further means that the specifications should be "functional" (The functional requirements need to be tested down the project stream). Another kind of minimality is obtained by use of a notation that supports us in saying just what we want, with no additional, redundant, or distracting verbiage. Verbose SRS do not gel well with software engineers—they would not want to spend a lot of time wading through superfluity. Redundancy can only add confusion and not certainty to the SRS document.

Verifiability is the capacity of determining that requirements are met at each stage of development. It depends on precise, unambiguous and concrete statements of constraints, exceptions and software functions. Unfortunately, verifying that design and code follow from specifications (SRS) is easier when specifications are procedural rather than functional. This puts a constant tension between the attributes of minimality and verifiability.

Transformability describes the characteristics of specifications that provide for a smooth processing to create a design, the working product of the next stage of the development life cycle. This smooth transition will minimize the amount of "rework" and adjustment needed to prepare for the design process (Effort on rework can be found by logging the time spent on the correcting the defects in the design.

document). For this reason, specification notation should be compatible with the methodology and tools used during design.

Modifiability refers first to the ease with which changes can be made to the specifications themselves. Modifiability is also promoted by modularity in the specification with minimum cross-connections and dependencies between different requirements.

Traceability is the inclusion of sufficient pointers, cross indexes, references etc to allow us to go forward from a requirement to find those part of a subsequent work product to satisfy the requirement or backward from a portion of the work product to find the requirements that give rise to it. Traceability also refers to our ability to correlate the specifications backward to the requirements analysis as shown in Figure 1 below.

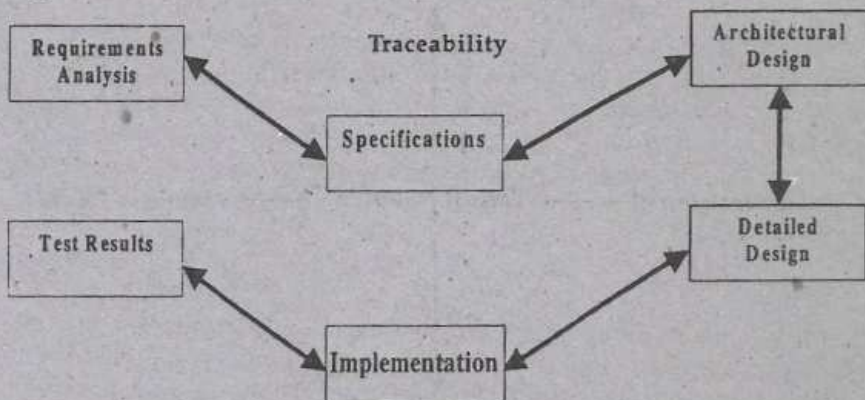


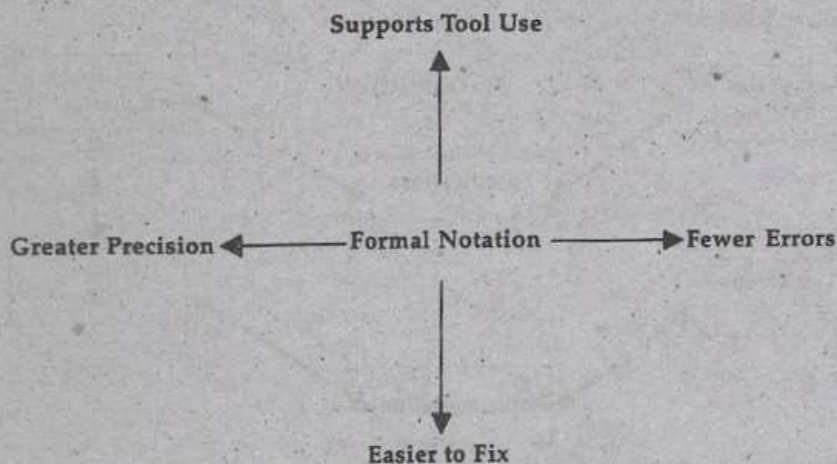
Figure 4: Requirements traceability to other work products

Formality, as indicated in the figure 4 above, is the key to much of the specification process. Software engineers often resent the time spent on documented specification, believing that it simply delays their "real" work i.e. coding. However, most of the essential attributes of good specification can be supported. SRS should be written in a language that is minimal—they include nothing that is not needed. A specification written using formal specification language is supposed to be characterized by the following:

- Appropriateness
- Cleanliness
- Constructability
- Structuring
- Ease of access
- Precision
- Lack of ambiguity
- Completeness
- Consistency

- Analyzability
- Formality
- Testability
- Traceability
- Executability
- Tolerance of incompleteness
- Adaptability
- Economy of expression
- Modifiability

Thus, the advantages of Formal Notation can be summarized as indicated below.



1.6 Quality Assurance in Software Support Projects

So far, we have discussed quality in the context of developing software products, but it is vital for software developers to recognize that the quality of support for a product is normally as important to customers as that of the quality of the product itself. Delivering software technical support has quickly grown into a big business. Today, software support is a business in its own right. Software support operations are not there because they want to be. They exist because they fill a vital void in the software industry; helping customers use the computer systems in front of them, a job that is getting more and more difficult. There is a phenomenal increase in the number of people who use their computers for "mission critical" applications. This puts extra pressure on the software support groups in organizations. We discussed the concept of "complexity measure" in the previous section. During maintenance phase of a software project, the complexity metrics can be used to track and control the complexity level of modified module/routines.

In this scenario, the software developer must ensure that the

customer's support requirements are identified, and must design and engineer the business and technical infrastructure from which the product will be supported. This applies equally to those businesses producing software packages and to in-house information systems departments. Support for software can be complex and may include:

- User documentation, including on-line help text
- Packaging and distribution arrangements
- Implementation and customization services and/or consulting
- Product Training
- Help desk assistance
- Error reporting and correction
- Enhancement

For an application installed on a single site, the support requirement may be simply to provide telephone and assign a staff member to receive and follow up queries. For a shrink-wrapped product, it may mean providing localization and world-wide distribution facilities, and implementing major administrative computer systems to support global help-desk services.

1.7 The SQA Function

One of the critical challenges for any quality program is to devise a mechanism for enabling independent reviews of the work done by others. The need is to focus on those SQA methods that permits development work to be reviewed by people who are not primarily developers. As per S/w CMM (Capability Maturity Model) version 1.1, Software Quality Assurance includes activities like AUDITING, REVIEWS etc. in addition to performing WALKTHROUGHS, PROCESS CHECKS etc. These could be performed by different sets of persons. A Person performing this role is called the "Software Quality Analyst". Thus, the acronym "SQA" is used in both senses of the terms (the role as well as the function in an organization). The role of a Software Quality Analyst is to monitor the methods and standards the software developers use and to verify that they have properly applied these methods to their work products. Quality Assurance as a function is a valid discipline in its own right. People can be quality assurance experts without being software design experts. This expertise and commitment of Quality Assurance function is what is required to establish a strong quality program in an organization. It includes knowledge of statistical methods, quality control principles, the software process, and an ability to deal effectively with people in contentious situations.

A key management axiom says that: "What is not tracked is not done." In software there are so many things that need to be done but the manager can't possibly track them all. Since effective management of

software project requires that thousands of activities are done precisely right, some mechanism is needed to do the tracking. This is the role of Software Quality Assurance function. Before establishing an SQA function it is essential to first decide how important software quality is to the organization, is it, for example, more important than meeting a critical delivery schedule? Just how much "quality" is important? Is it looked upon merely as a means to woo customers? Should the product be delayed to fix 1 more bug, or 10 more, or 100 more or...?

As long as software quality is treated as a matter of faith, no formal actions are required. Incantations about the need to believe in quality will change nothing. If management's commitment to software quality does not come down to day-to-day actions, no consistent improvement is likely.

Similarly, if management views the mere act of setting up a Quality Assurance just to show the presence of such group, then, it will have wasted a lot of money, established another bureaucratic bottleneck, and probably used to be effective. The prime benefit of establishing a dedicated Quality group is the assurance it provides management in showing that the officially established process is actually being implemented. More specifically, establishing a Quality Assurance program in an organization ensures that:

- An Appropriate development methodology is in place.
- The projects use standards and procedures in their work.
- Independent reviews and audits are conducted.
- Documentation is produced to support maintenance and enhancement.
- The documentation is produced during and not after development.
- Mechanisms are in place and used to control changes (Configuration Management).
- Testing emphasizes all the high-risk product areas.
- Each software task is satisfactorily completed before the succeeding one is begun. (Phase-end Inspections)
- Deviations from standards and procedures are exposed as soon as possible. (Waivers and Deviations)
- The quality control work is itself performed against establishment standards.
- The SQA plan and the software development plan are compatible.

1.7.1 The Benefits of SQA Function

The reasons for concern about software quality are compelling. Especially, these days, in the face of the American WTC tragedy, everyone who travels in an airplane agrees on the importance of quality software for air traffic control and the quality of security measures employed on airports. Similarly, a payroll processing software should not have errors, nor should the software that controls air defenses, flies aircraft,

steer ships, runs banking systems forecasts weather, directs police, and so on. Software has an enormous impact on almost everything we do, and this impact will only increase in the future.

"SQA" is used in two contexts; as a role (Software Quality Analyst) and a function (Software Quality Assurance). Beyond logical arguments on the need for SQA, experience also provides a useful guide. In a study, aimed at studying software managers' views of the reasons for project success or failure, it was found that while the value of quality Assurance was not explicitly addressed, when the project management standards were enforced, 76 percent of the projects were successful, as opposed to only 60 percent when no standards were followed.

The experience with IBM system/360 and 370 software development also demonstrate the value of SQA in enforcing development standards. IBM reported software quality improvements of three to five times over an eight-year period. While this was due to many factors, SQA was an important part of their project. Many project managers feel that it is found increasingly advantageous, from both product quality and cost-effectiveness standpoints, to have an explicit quality standards enforced on their software projects. Thus SQA has significantly increased ability to predictably engineer reliable systems.

1.7.2 The Need for Software Quality Assurance Function

It is hard for anyone to be objective about auditors- this is a normal human psychology. People feel that they do their jobs pretty carefully and as such they resent any contrary implication. The need for "outside" review, however, is a matter of perspective. Suppose you had just packed a parachute and were about to take a jump. The odds are you would be happy to have a qualified inspector monitor your every step to make sure you did it just right. Similarly, when quality is a vital business issue, some independent checks are necessary, not because people are not trustworthy but because they are human. There is a well-known saying that "to err is human". The issues with software are not whether checks are needed, but who does them and how. The people employed in the Quality Assurance function perform this independent check.

In very small organization it is often possible for the software managers to monitor the work so closely that no SQA activity is needed. As the size of the staff grows, the managers become involved with other duties, and they quickly lose touch with the day to day technical work. This is when they need to do one of the following;

- Find some way to handle their own workload so that they can monitor more closely their people's work.
- Hire someone to do the audit work.
- Motivate the people to monitor each others' work

From technical, economic, and moral viewpoint, the last alterna-

nonoccurrence if the appropriate standards and procedures have not been reasonably met.

- If SQA fulfills its responsibilities and if senior management refuses to allow line management to commit and to ship products until the SQA issues have been addressed, then the SQA can help management improve product quality. Items typically brought under SQA review are shown in Table 3.

Table 3: Example of Items under SQA Review

-
1. *requirements traceability matrix to show that the product specifications cover the requirements asked for by the customer*
 2. *implementation traceability matrix or similar tool used to show that the product specifications are implemented in the design*
 3. *documentation samples*
 4. *Samples of development records*
 5. *Software Configuration Management activities as per practices stated in the configuration management plan, change control board*
 6. *Sub-contractor's quality assurance function (important when outsourcing part of or entire development work)*
 7. *All plans prepared during the tenure of software projects*
 8. *Peer Reviews and Testing related activities*
 9. *Reviews of design, documentation and code to see adherence to organization's standards*
-

Since the potential workload is enormous, some of these reviews are generally done on statistically selected sample of the work. Next chapters deal specifically with other areas and aspects of software quality assurance.

REFERENCES

- [1] *Software Engineering: A Practitioner's Approach*—Roger S. Pressman (McGraw-HILL Publication)
- [2] *Software Quality Assurance & Management*—Michael W. Evans, John J. Marciniak (John Wiley & Sons)

Chapter 2

MANAGING SOFTWARE QUALITY IN AN ORGANIZATION

2.1 Quality Management System in an Organization

In chapter 1, we discussed importance of quality and how quality can be brought out at each level of the software systems development life cycle. In this chapter, our focus will be on understanding quality management in an organizational context. We'll discuss the details of a quality management system, contents of a Software Quality Assurance Plan and roles and responsibilities of the Software Quality Assurance (SQA) group in an organization that has committed itself to quality system.

Applying the principles of quality to the software process is the beginning of organization's commitment to quality. An organization expresses its commitment to Quality through an apex document known as the 'Quality Policy'. The term QMS (Quality Management System) is used internationally to describe a process, which ensures and demonstrates the quality of the products and services provided by the organization. QMS is a set of procedures/processes, which as a whole form the basis for executing organization's product/service delivery mechanisms. Typically, the Quality Manual begins with a Vision and Mission Statement of the organization committing itself to quality; for example, a vision statement can read like "To be the leader and most respected e-Solutions Company in India", or say "Providing best of the breed CRM solutions to our customers". Organization's Vision is followed by its Mission Statement. Figure 1 shows the Quality Pyramid with the terms mentioned above.

An example of a mission statement could be "We are in the business of Providing Value to our customers through CRM solutions with a commitment to quality of solutions providing. Making our organization a winner in the Global e-Business Market with solutions that are cost effective and with requisite quality". Having decided on the Vision and Mission for the

business, an organization must, next, device a mechanism that helps deliver the solution with the commitment made. QMS, which is a set of well thought out and well defined processes helps to do this.

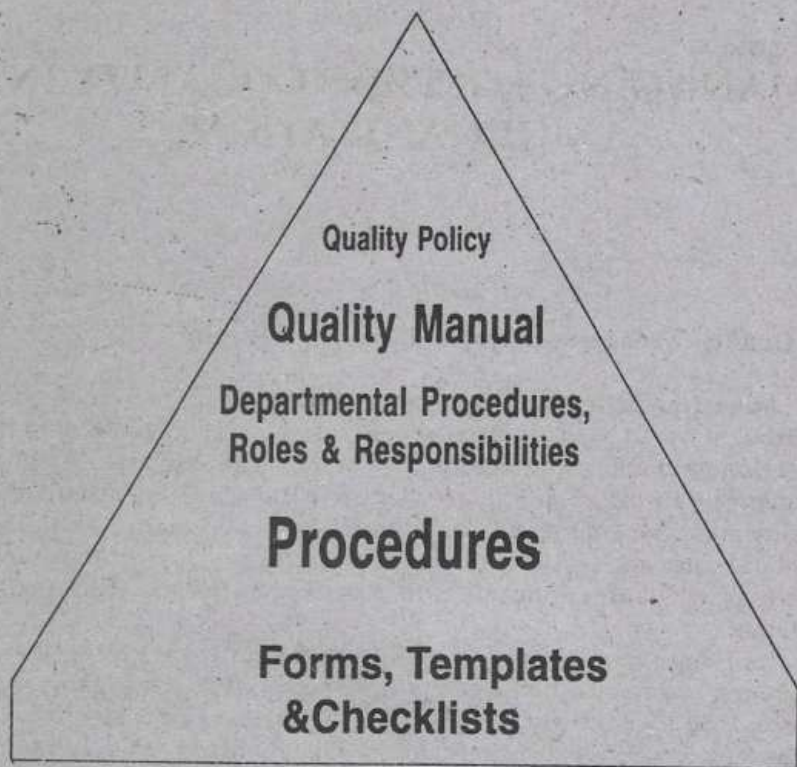


Figure 1: The Quality Pyramid

Typical contents of the quality manual are:

- Introduction (Purpose, Definitions, References)
- Organizational structure—various roles for software delivery (Project Management structure, structure of the Quality Group along with the descriptions of various sub-groups within it such as—
 - Process Engineering Group (PEG)
 - Software Quality Assurance (SQA)
 - Measurements Assurance (MA)
 - Defect Prevention (DP)
 - Technology Change Management Group (TCMG)
 - Quality Systems Requirements (Quality Policy, Quality System Procedures, Approach to Quality Planning along with Quality Objectives)
 - Software product/services realization mechanism deployed in the organization

2.1.1 What is QMS

The ISO 9001:2000 is one of the International Standards, which, defines and describe what is required of a satisfactory quality system. It specifies requirements for a Quality Management System (QMS) where an organization:

- a) Needs to demonstrate its ability to consistently provide the product that meets customer and applicable regulatory requirements, and
- b) Aims to enhance customer satisfaction through the effective application of the system, including processes for continual improvement of the system and the assurance of conformity to customer and applicable regulatory requirements

(ISO 9001:2000 Clause 1 Scope—In this international standard, the term “product” applies to product or service intended for, or required by, a customer)

Apart from ISO, many other industries, national and international bodies have evolved standards, which describe quality systems to be applied to software development and support. These are discussed later in this chapter. (In addition to this, we have devoted a separate chapter on ISO 9001:2000 and also a separate chapter on other Software Process Improvement frameworks)

The term ‘quality system’ is sometimes used instead of ‘quality management system (QMS)’. This emphasizes the need for the quality process to be actively managed to ensure that it continues to be effective and efficient. Software development organizations must keep the practices and tool they use under constant review and make changes in a controlled way. This helps to ensure that the QMS is effective in the face of continuously changing business environment.

Just as important as the practices and tools are the organization’s employees who use them. They need to be properly trained (through awareness programs etc.) in usage of the QMS. Training plays a very important part in quality related activities of an organization. It is essential that a quality professional has the mindset and skills for imparting training within his/her area of responsibility. The onus goes to the quality system as well. QMS must ensure that the employees have the right skills to do their jobs in a professional way. If they need training, they should receive it (with proper metrics to evaluate effectiveness of training imparted). QMS must also ensure that employees understand their responsibilities and how their work relates to that of others.

Successful QMS gives great emphasis to early corrective action. It is much cheaper for the software developer to correct errors early in development life cycle (We discussed ‘cost of quality’ in Chapter 1).

It is also worth realizing that an error might cost the customer much more than it costs the developer. The aim must be to inculcate 'right first time' attitude throughout the organization. Quality Control activities such as *Inspection* and *Testing* are therefore to be built at every stage to detect errors as early as possible before moving on to the next stage of building the product. (Inspection and Testing is also separately discussed in chapters devoted to these topics).

It is always better to avoid making errors in the first place. Successful QMS include ways to analyze records and errors to determine their causes and make action plans to prevent errors by eliminating their causes (*Root Cause Analysis*). QMS must assure customers and developers that software products and services developed by adhering to the QMS will be of good quality.

QMS should be auditable. This means that the development process must be well documented and quality records including suitable measurements must be generated throughout the software development process. Quality Record is a record kept as part of QMS—It is a document, which furnishes objective evidence of activities performed or results achieved. For example; review record of a Project Plan, Code Review results, Test Execution Report etc. Quality records help demonstrate the achievement of quality and the effective operation of the quality management system. Many persons and organizations resist to QMS implementation because they feel it leads to paper factory. However, this is a misconception. The overheads of such record keeping need not be excessive (it depends how practically the QMS is designed) if the quality system is carefully designed. The improvement in quality more than compensates for the costs and efforts involved.

As per ISO 9001:2000, general requirements from Quality Management system are that the organization shall establish, document, implement and maintain a quality management system and continually improve its effectiveness. In particular, ISO requires that the organization shall

- a) Identify the processes needed for the quality management system and their application throughout the organization
- b) Determine the sequence and interaction of these processes
- c) Determine criteria and methods needed to ensure that both the operation and control of these processes are effective,
- d) Ensure the availability of resources and information necessary to support the operation and monitoring of these processes,
- e) Monitor, measure and analyze these processes, and
- f) Implement actions necessary to achieve planned results and continual improvement of these processes.

Processes needed for the QMS referred to above should include processes for management activities, provision of resources, product realization and measurement. Figure 3 depicts this concept.

Software development can be exceedingly complex and there are often many alternative ways to perform the various tasks involved in the software development lifecycle. A defined process can help guide the software professionals through these choices in an orderly way. Need for Process Standards and Definitions can be put forth as follows:

- Process standardization helps to reduce the problems of training, review, and tool support
- With standard processes/procedures and methods, each project's experiences can contribute to overall process improvement
- Process standards and definitions provide the basis for process and quality measurements
- Since process definition takes time and effort to produce, it is impractical to produce new ones for each software development project.

The conflicting needs for customization and standardization can often be resolved by establishing a process architecture, which consists of a standard set of process steps with rules for describing and relating them. Customization is then achieved through appropriate interconnections of these standard elements into tailored process models.

Thus, the philosophy underlining successful QMS is that the continuous improvement of every aspect of the software process. The quality records and measurements are analyzed and used for this purpose. (For example comparing defects per KLOC - Kilo Lines of Code of a project with that of the organizational standard and seeing if the performance lies within the upper and lower control limits defined). In the light of this analysis, senior management must regularly review (through periodically repeated audits and assessments) the effectiveness and efficiency of the quality management system, and ensure that action is taken to improve it.

In summary, a QMS is a set of procedures. Procedures are at the heart of the quality system. They are the documents, which tell staff how to follow a quality system, day to day, in their work. QMS is management's means to establish a uniform and consistent approach to product realization/providing services. The quality system is the complete work process, including policies, procedures, tools and resources, both human and technological as shown in Figure 2.

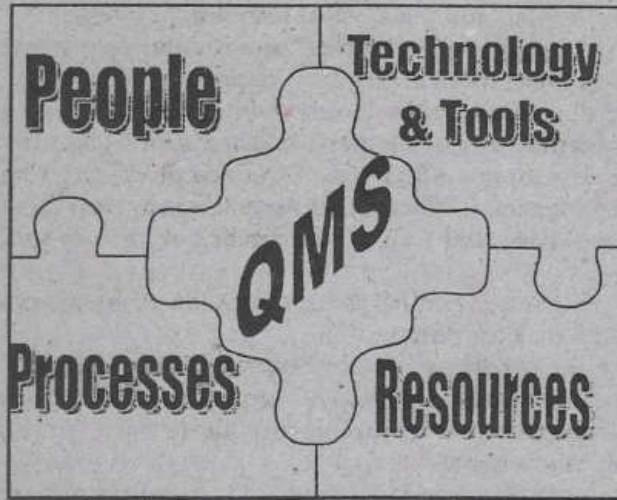


Figure 2: Components of a Quality Management System

2.1.2 Quality Policy

Quality Policy is a document that states the overall intentions and direction of an organization with regard to quality as formally expressed by top management. Clause 5.3 of ISO 9001:2000 says that

Top Management shall ensure that quality policy

- a) is appropriate to the purpose of the organization (i.e. Suitable for the business conducted by the organization)*
- b) includes a commitment to comply with requirements and to continually improve the effectiveness of the quality management system,*
- c) provides a framework for establishing and reviewing quality objectives,*
- d) is communicated and understood within the organization, and*
- e) is reviewed for continuing suitability*

The implied meaning of this requirement is that Quality policy must be documented. Quality policy does not have to include objectives but should create a framework for establishing them. Quality policy has to be assessed at its merit in relation to Management Commitment and Continual Improvement. Figure 3 illustrates a model for process based QMS.

Top management should begin the quality program by forming a quality policy. Quality Policy should be a clear statement of the organization's commitment to quality, and management's expectations of the quality program. The Quality Policy should be published, and communicated to the employees so that it understood and implemented at all levels in the organization.

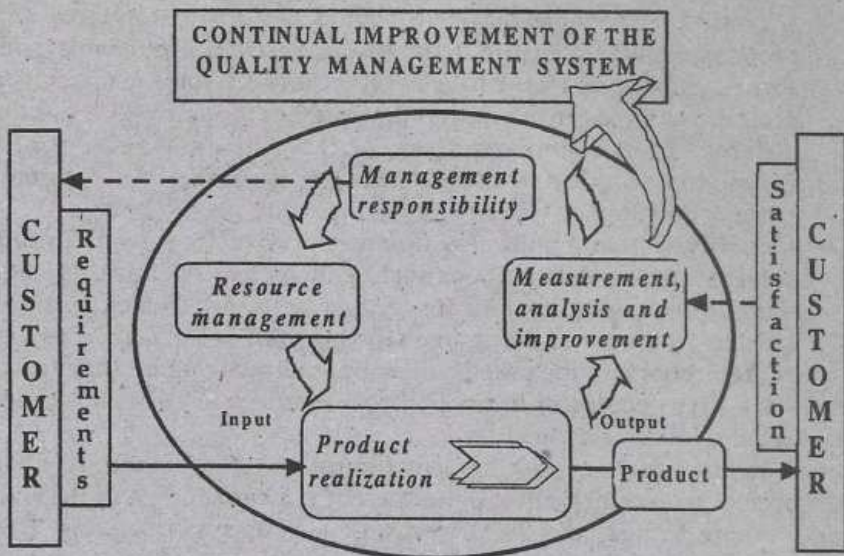


Figure 3: Model of a process-based quality management system

2.2 Quality Management System: Various Expectations

A quality management system is used by a number of users within and outside the organization: Senior Management, Project Managers, Programmers, Application Systems Designers, Analysts, and Staffing department, Testers, Marketing Department. Each of them has certain expectations from the Quality Management System deployed in the organization.

2.2.1 The Project Manager

Project manager, in this, book, is referred to as the member of staff who has day-to-day control of a software project. He is the person armed with responsibilities of monitoring the project by checking its progress, innovating by introducing new methods of working; represent his or her team while dealing with the customer. Here are some of the ways in which, a Quality Management System is expected to support these functions of a Project Manager:

- A quality system should provide facilities whereby a project manager can consider the vast majority of possible risks, which could affect a project.
- A quality system should specify standards, which will enable staff to report on their activities in a uniform way (formats for monthly status reporting). Such reports should be able to be easily processed by a project manager.

- A quality management system should lay down standards and procedures, which ensure that cost and expenditure records from earlier projects are kept in a easily accessible form. This way, a project manager can check that the estimate for a project is not too different from a similar project, or more likely, that parts of a system to be produced have similar costs to those parts of systems already developed.
- A quality system should provide a way of collecting and analyzing defect statistics; data on bugs such as their classifications, average life in queue, severity, and trends and patterns. This data can be used in a number of ways: in analyzing staff performance; as one of the inputs into the process of evaluating whether an innovation has been a success; or in predicting the pattern of errors in a product when it is released.
- A quality management system should contain a standard for a project plan which gives details about the capabilities of the staff who are to carry out the project task such as system testing. This enables the project manager, while representing the project to a concerned department for resource request.

This is just a subset and not an exhaustive list of the facilities that should be provided by a quality management system.

2.2.2 The Programmer/Developer

Programmer or Developer is a member of staff who is given a specification for a chunk of software and who then has to produce program code, which is taken through unit testing. Most programmers also carry out modification to the code as requirements keep evolving. Some of help that a quality system should provide to the developer is:

- The quality system should set coding standards, which determine the way that a software program is to be constructed. Use of coding standards, guidelines make testing and debugging easier
- It should provide directions to the developer to store away test data and test outcomes in files. This way, re-programming becomes easier. The developer can retrieve test data, modify it in response to, say a requirements change, and then apply it again.
- It should provide a standard which insists that the names of the software chunks, which have been produced by a developer, have a correspondence to the files used to store the source code, object code, test data and test outcomes for the software. In other words, the QMS should guide the developers and the configuration manager towards organizing the entire software project repository. Typically, the items in a project repository are the following:

- Software Requirements Specification Document (SRS document)
- Design/Application Architecture Document
- Change Requests
- Coding Standards
- Quality Documents
- Test Procedures
- Test Cases and Test Scripts
- Test logs
- Source Files
- Binary Files
- Model files, Property Files

2.2.3 The Application System Designer

A systems designer is responsible for preparing a description of the software application/system that is to be produced. He/she is the design architect and in this role his/her job is to prepare the requirements specification or use the given requirements specification to produce the system design. He/she provides architecture which implements the functionality demanded by the customer. A number of the facilities that a good quality system should offer the system designer are shown below:

- It should provide a standard, which describes how the process architecture and data architecture are to be written. A good standard should enable the designer to check that the system, which has been, designed actually implements the functions described in the requirements specification.
- It should also specify standards, which lead to a system that should be easily maintainable. For example, a good standard for design is high cohesion and low coupling. (Cohesion means insisting that each module in a system carries out one function—an only one function.. Reducing the coupling between modules means, in effect, reducing the complexity of the connection(s) between the modules as much as possible.) For details of this fundamental design principle, reader is referred to Ref [1] Design based on high cohesion and low coupling principles tend to be easier to maintain than a system consisting of modules that carry out a large set of functions.
- It should provide standards, which lead to a good requirements specification. For a system designer, the word 'good' implies that the requirements specification should contain no ambiguities, contradictions or platitudes, and that constraints such as response time are directly cross-referenced to the functions to which they apply.

2.2.4 The Analyst

An analyst is a member of staff who maintains a liaison with a customer to produce requirements specification which is used by system designer to produce a blue print for carrying out the functionality intended by the customer. The analyst's need is centered around the requirements specification. Some examples of how a good quality system can help him toward this is shown below:

- It should provide a standard for the requirement specification, which, among other concerns, specifies an organization of the requirements specification in which related functions are contextually close to each other. For example, in a stock control system, the descriptions of the functions associated with querying database information about products, which are out of stock, should be physically close to each other. This makes it easier for the analyst to check for contradictions.
- It should provide a number of checklists, which enable the analyst to ensure that features or issues associated with a software system that is to be developed are not forgotten. Such checklists are usually general.
- It should provide a description of the processes involved when an analyst liaisons with a customer; for example, specifying the way in which functions in the requirements specification are checked against the customers requirements.

2.2.5 Senior Management

Senior management comprise of the members of the organization who work at the next level of the management structure above project managers. They are also called business mangers or program managers, and usually supervise a number of projects. Senior management members will require the quality system to provide a number of facilities; some of which are enumerated below:

- It should provide senior mangers with reports of achievements against targets for each of their projects; for example, what tasks should have been completed by a certain date and what tasks have, in fact, been completed, what expenditure should have been incurred by a certain date and what expenditure has actually been incurred i.e. Earned Value Analysis (See the Vignette below).

"Earned value" is a management technique that relates resource planning to schedules and to technical cost and schedule requirements. All work is planned, budgeted, and scheduled in time-phased "planned value" increments constituting a cost and schedule measurement baseline. There are two major objectives of an earned value system: to encourage contractors to use effective internal cost and schedule management control systems; and to permit the customer to be able to rely on timely data produced by those systems for determining product-oriented contract status.

- It should provide direction on the setting up of audit trail documentation. Such a trail might consist of memoranda, which have been generated by checklists; for example, when analysts in a project have queried a particular set of requirements with the customer. Such audit trails are useful for members of senior management; for example, when resolving some dispute about requirements, which has escalated out of a project to his or her level of responsibility.
- It should provide facilities whereby reports on defects discovered during development are issued regularly. Such reports are not only of value to the individual project manager, but are also useful for senior management. For example, where a senior manager is actively involved in a management team which is attempting to carry out some *process improvement*, diagrams showing where defects were introduced during development, and where they were discovered, would be a major input into the tasks of determining processes that need improvement. (Frameworks for "Process Improvements" are covered in a separate chapter devoted to this topic)

2.2.6 Staffing Department

Staffing department is a central function responsible for providing human resources to the projects. Sometimes, this department may also be entrusted with the responsibility of training. There are a number of ways in which a quality system could serve the interests of this department; for example:

- The standards for a project plan should insist that the skill level of the staff on the project is properly specified. This obviously helps the staffing department to allocate the right people to the project, and to specify requisite training if staff with the specified skills are not available. The sub-clause 6.2.2 of clause 6 (Resource Management) from ISO 9001:2000 specifying this requirement is quoted below.

6.2.2 Competence, awareness and training

The organization shall:

- d) determine the necessary competence for personnel performing work affecting product quality,
- b) provide training or take other actions to satisfy these needs,
- c) evaluate the effectiveness of the actions taken,
- d) ensure that its personnel are aware of the relevance and importance of their activities and how they contribute to the achievement of the quality objectives, and
- e) maintain appropriate records of education, training, skills and experience

- The quality management system should provide instructions on the activities carried out when the project has been completed. One activity relevant to a staffing department, which also has a personnel function, is for the project manager to produce a written report on the effectiveness of the staff on the project. After all, as a result of working on the project, the members do value addition to their profile. It is important that their updated resumes get submitted to organization's human resource database so that it is useful to the next project to which they are to be allocated. Quality Management System should provide a standard format for this report as well as for the resume.

2.2.7 The Customer

The customer is also the user of the quality management system of the supplier organization. The customer's concerns are rather different from those of the developer; for example, he or she is concerned that adequate means are going to be used to develop the software and to check that it meets requirements; that adequate reporting facilities are provided which enable the progress of the project to be monitored, and that adequate liaison occurs during the period in which the developer is demonstrating that the system actually meets user requirements. (They say "QFD—Quality Function Deployment is a technique to incorporate "customer's voice" into the design. Ref [3] has a discussion on this.) Some examples of the type of facilities that a quality system should provide for the customer are:

- It should provide directives which specify how progress meetings are to be organized: who should be invited from the developer's staff, when the meetings should be scheduled, what physical arrangement, such as the booking of a room, should be made, and how issues arising from a progress meeting are to be resolved.

- It should provide guidelines about how facilities provided are to be extracted and used as quality controls for a project. This list of controls would be specified in the quality plan. This is often a contractual document (also called Document of Understanding), or is at least signed off by the customer. This document should be complete description of the way in which the developer intends to check that the system meets customer requirement, and the role of the customer in this process.
- It should specify the format of reports that are to be sent to the customer concerning progress and how to determine the frequency with which these reports are sent.

The sub-clause 7.2.1 of clause 7.2 (Customer Related Processes) from ISO 9001:2000 is quite related to these customer concerns. It is quoted below.

7.2.1 Determination of requirements related to the product

- a) The organization shall determine requirements specified by the customer, including the requirements for delivery and post-delivery activities,*
- b) Requirements not stated by the customer but necessary for specified or intended use, where known,*
- c) Statutory and regulatory requirements related to the product, and*
- d) Any additional requirements determined by the organization*

2.2.8 The Testers

System Testing and Acceptance Testing (These terms are discussed in the chapter devoted to Testing) are the two critical activities in a software project. Either the development staff or people from quality assurance department derive a series of test cases from the requirements specification. The intention is to check that the system, which has been developed, meets customer requirements. Such staff would expect a number of facilities from a quality system:

- It should ensure that the *requirements specifications* are constructed in such a way that system tests can be easily derived and related back to the functions they test. Quality Management System should also ensure that procedures and associated checklists are in place to enable test data and test outcomes to be stored in a project repository. This is called the "Re-Use Pack". The test files in the Re-Use Pack can easily retrieved when re-testing takes place.
- QMS should ensure that procedure exists to ensure that those responsible for test planning can check that adequate resources are available during the test phase; for example, that during the

development of a software system integration, software and hardware required for simulating the test environment are actually available.

2.2.9 The Marketing Department

The job of the marketing department is to sell the software systems and applications prepared by the organization. In an organization, which is in the business of developing one-off or tailor made systems for the customers, the effectiveness of the marketing department will determine the profit levels. A quality system is expected to offer facilities to staff involved in marketing. For example, as part of the project debriefing process, every project should produce a list of sub-systems and modules, which can be used in the next project. Based on this list, in the next bid, the marketing department, along with the project staff may be able to produce estimates to the client. Being backed by the strong learnings and re-usable components, in the next project, the organization is likely to be more productive and smarter in bidding and thus it can have edge over its competitors. A Quality System should encourage the projects to document their "lessons learnt" by providing a repository, mandating metrics on project productivity (Effort logs), schedule slippage data etc.

Thus, in this section, we have discussed examples of how various users in the organization expect the Quality Management System to help them. In summary, every one in the organization is the user of the QMS, and hence in a software company, every one is responsible for quality assurance.

2.3 Quality Assurance: Some Diagnostic Questions

Given such wide range of expectations from the Quality Management System, it must get evaluated for its effectiveness. One of things expected from a Quality System Auditor is to visit a client organization and evaluate its quality system for adequacy (this is known as "Adequacy Audit"). Before embarking upon the final audits by an third party auditing organization, it is important to know whether the company wishing to go for quality certification has an adequate quality system in place. There are few typical questions, which pinpoint problems in quality. Most of them are directed at the software developers of the client organization. There are two aims in discussing these questions here: one is to make you appreciate how to evaluate your quality system and, second to have an introduction to the rest of the topics discussed in this book. These diagnostics questions are:

- Functional traceability
- Project monitoring

- Design validation
- Quality planning
- Quality system review
- Unit test records
- Planning for systems testing and acceptance testing
- Risk analysis
- Configuration management practices
- Training
- Intergration testing
- Project costing procedures
- Cost checking and tracking
- Early requirements checks
- Test data storage
- New technology impact analysis
- Traceability into testing
- Decision making rationale
- Standards and procedures for auditing
- Functional specification structure
- Defect data analysis
- Design guidelines
- Acceptance test failures
- Project debriefings

Let us now discuss each one of above points briefly.

2.3.1 Functional Traceability

This is one of the first few questions that concern traceability. Ask a developer to show some of the listings of the program code of a completed project. Indicate a module and ask him or her to explain which requirements (as stated in the Requirements Specification Document) that module has implemented. If they can point out to the functions/specifications, then it is a good sign that probably the organization has a good quality system. If this takes a long time to answer, you can suspect that probably there is something wrong with the organization's quality system or it may even be non-existent.

2.3.2 Project Monitoring

It is a good idea to ask this to a project manager—"how do you monitor the progress of your project?" (i.e. in terms of tasks completed, tasks that slipped and the amount of resources expended at a certain point in a project as compared with the amount of resources planned at the start of the project.) If the project manager can point out to procedures, which enabled him or her to get this information easily, then the part of the quality system dealing with planning and project monitoring is in place.

It means that the planning is carried out at a task level, and there are good facilities for staff to report on task completion. This is a topic on its own—Project Planning and out of the scope of this book. Ref [4] indicated at the end of the book is one concise and good reference to Project Management that reader may like to refer.

2.3.3 Design Validation

It is worth asking a developer what efforts he or she takes to validate a design; i.e. what quality controls exist to provide assurance that a design meets its corresponding requirements specification—both functional and non-functional. If the answer you receive is that a senior staff member signs off the design, then it is not adequate. If the developer states that method such as *technical reviews* and *prototyping* are used and that standards and procedures exist for these activities, then it is an indication of quality system existing in the organization. Reviews and Inspection are discussed under chapter 6 of this book.

2.3.4 Quality Planning

A very relevant question to be asked to the developer is about the quality plan of his project. If the developer simply states that each project uses the quality manual of the organization, then it is a poor answer. The correct answer is that the organization has a quality manual, but that the elements of the manual relevant to a project are extracted after quality factors have been identified and a risk analysis has been carried out.

2.3.5 Quality System Review

Another useful question to developer is: how does the organization evaluate the effectiveness of the quality system and also, how does it evaluate the potential changes that new software development technology and business policy might have on the quality system? A quality system is an evolving entity, and it is important that mechanisms are in place to evaluate what degree of evolution should be. An organization should receive information such as project debriefing reports, error statistics, and project audits, or periodical project reviews. These should be used as input into the process of evaluating the effectiveness of the quality system. (Quality Systems Improvement)

2.3.6 Unit Test Records

An important question concerning *Unit Testing* is what is the entry point to begin unit testing and what records exist to prove that a programmer has actually carried out a test? Many quality systems do not provide

enough standards for this part of the software project. If it is not obvious that a programmer has carried out testing, but that all that might have happened is that a clean-compiled module has been passed off to staff who carry out the integration, then one can say that there is a weakness in the quality system; it allows developers to pass untested modules to staff who carry out *System Testing* and *Acceptance Testing* at a later stage in the project.

2.3.7 Planning For Systems Testing And Acceptance Testing

Another revealing question to ask about system and acceptance testing is: when does your quality manual insist that you start planning and thinking about system and acceptance tests? If the answer is that this occurs during later stages of the *System Development Life Cycle* (SDLC), say, during coding, then it is not a good situation. The correct answer is that as soon as the requirements specification has been validated as being correct, the system tests and acceptance tests should be developed in outline.

2.3.8 Risk Analysis

Another question concerned with project planning is: what standards and procedures does your quality manual contain to deal with the identification of risk? All projects are risky, with a degree of variation from one to another. A good quality management system should provide standards and procedures, which direct the processes of organizing and planning a project based on an appraisal of the possible risks in the project. A quality manual should provide procedures for analyzing the risks in a particular project and guidelines, which provide advice on how to minimize the effect of risk.

2.3.9 Configuration Management Practices

It is worth investigating how the process of change is managed. A typical question that can be asked in this regard is: what happens when an error is discovered during operation and it necessitates changes to the requirements specification, system design and program code. This is a question about configuration management; the process of documenting, appraising and implementing changes to a system. In an organization with a reasonably good QMS, the developer is expected to point to standards and procedures which describe how errors are notified to the software developer; how any proposed changes arising from the error are communicated to staff in charge of configuration management; how a decision to carry out the change is made, how the change is applied and validated; and finally, how staff who should know about the change have details communicated to them.

2.3.10 Training

A good project ensures that its members are qualified and experienced enough to carry out project tasks assigned to them, or at a minimum, they have been trained adequately in that task. If an organization does not keep adequate records of training, then a project manager is unable to access important information needed for planning. The question to ask in this case is: do you keep training records? Can they be evidenced? It is worth looking at these records to see what training is considered worth by the organization: if project team members are just being sent for courses in programming languages alone, then this is not a very good sign. However, if the project team members are sent on a variety of software engineering related courses on topics such as requirements specification, system design, project management, quality assurance and testing, then that is a much better sign.

2.3.11 Integration Testing

Integration testing is a term used to describe the process of validating a system as it is being built up incrementally, with coded modules being added a few at a time. Developing a system this way enables errors to be discovered more efficiently than if the system was not integrated and just collected together prior to system testing. A good question to ask is: have you got standards and procedures to describe the process of integration testing, which, for example, provide advice on integration strategies and describe the documentation to be generated from integration testing?

2.3.12 Project Costing Procedures

A good question to ask is: are there standardized procedures, which every project has to use to use in establishing a cost estimate? It is important that projects use the same costing technique, as the results from previous costing can be used for future projects. This may not seem to have much to do with quality; however, projects which make erroneous costs usually start running out of budget during testing and often have to cut corners on the testing process, thus delivering a poor quality product.

2.3.13 Cost Tracking

It is important that there are standards and procedures, which enable a project manager to check the estimated cost of a project. Probing questions should be asked about the existence of such standards and procedures. For example, are there mechanisms whereby a project manager can prepare the cost of part of a finished system which is similar

to part of a proposed system by looking at historical data on costing, or even the actual amount of resources spent on the finished system?

2.3.14 Early Checks On Requirements

Another good question to ask an organization is: what is the earliest point in a software project at which a system and its documentation are checked against the customer requirements? The answer to this should be that the requirements specification is checked just before it is frozen, and the developer has clarity on it adequate enough to proceed with coding. The validation technique could be such as a requirements review or prototyping. Everyone knows that the earlier the errors get detected in a life cycle of a project, the cheaper it is to fix them.

2.3.15 Test Data Storage

There is a question you can ask to find out discover how serious an organization is about systems maintenance: do your standards insist that, as far as possible, test data for module, integration, system testing and acceptance testing should be stored? This is what we had referred to as the "re-use pack". It ensures that if re-testing occurs, then little extra effort is expended in the re-developing old test data.

2.3.16 New Technology Impact Analysis

New technology can give rise to new demands on a quality system. The purpose of Technology Change Management is to establish procedures to identify, evaluate, select pilot and deploy newer technologies, which will foster quality process improvement. It is good question to ask the developers as to how the growth of new technology is monitored in their organization and how is it ascertained if the new technology threatens to affect quality system. One of the factors to be considered for quality systems improvement is its response to a number of factors; change of technology being one of them. You can also ask the developers if there is process in place for managing the changes in technology. For example, does the organization have guidelines for designing human-computer interface?

2.3.17 Traceability Into Testing

As an auditor, you can ask a question, which pinpoints problems in testing: can you easily trace from a function, expressed in your requirements specification, to the individual tests, which check out that function? In short, do you have a "requirements traceability matrix"; i.e. a two-dimensional grid chart that lists out the REQUIREMENTS on one hand and the FUNCTIONALITIES as the other dimension. If the

organization can readily retrieve these tests from past project documentation, then there is a good chance that the part of the quality system concerned with testing is in good shape.

2.3.18 Decision Making Rationale

An important thing in a software project is that when a major decision is made, it should be made with a full justification; i.e. the responsible person provides a rationale, or set of reasons, for making the decision in the way that it was made. You can take the developer into confidence to find out if standards for project organization, application system design and module testing include a directive, which specifies that the staff carrying out these activities provide this rationale. For example, a designer can be asked to explain why a particular design was adopted for a system. By getting these explanations from the development staff, a quality system encourages them to think more deeply about a task. This is very important from the CMM-I requirement angle where "Decision Analysis and Resolution" (DAR) is one of the process areas.

2.3.19 Auditing Standards And Procedures

A question which often enables a quality consultant to find out how seriously an organization takes quality is: do you have explicit standards, procedures and guidelines which govern the process of checking that projects follow standards and procedures? In some organizations (mostly large organizations with 500 plus employees in a single location), this is achieved through monthly reviews of the projects with a focus on finding out if each project in the location is following the processes defined in the organization's QMS. If such procedures exist, then they cover the process of auditing, when to audit, how frequently to audit, what reports are generated from the auditing process, and what actions to be taken when a project fails an audit. It may so happen that the Quality Manual omits any mention of auditing but this is not desirable.

2.3.20 Functional Specification Structure

It is worth exploring the standards that a developer uses for the requirements specification. A good question to ask is whether the functional part of the requirements specification is structured in such a way that all functions, which are connected with each other, are physically adjacent. A requirements specification organized in such a manner is much easier to access by both software development staff and the customer and, hence would be prone to fewer errors.

2.3.21 Defect Data Analysis

Validation of software systems gives rise to defect data, which describes the errors that have occurred, and their classification. A good question to ask is what happens to this data? Is it, for example, used to improve the technical process employed by the software developer and also the quality system?

Watts S. Humphrey, in his classic book "Managing the Software Process", provides the following classification:

Errors are human mistakes, and their quantification often depends on an understanding of the programmer's intentions. In the case of typographical or syntactic errors, their causes are generally clear, but the nature and cause of design errors is much harder to establish precisely.

Defects are improper program conditions that are generally the result of an error. Not all errors produce program defects, as with incorrect comment or some documentation errors. Conversely, a defect could result from such nonprogrammer cause as improper program packaging or exception handling

Bugs A bug (or fault) is a program defect that is encountered in operation, either under test or in use. Bugs result from defects, but all defects do not cause bugs (some are latent and never found). With widely used software, the defects may be found many times, resulting in duplicate or multiple bugs per defect.

Failures A failure is a malfunction of a user's installation. It may result from a bug, incorrect installation, or a communication line hit, a hardware failure, and so forth.

Problems are user-encountered difficulties. They may result from failures, misuse or misunderstandings. Problems are human events as opposed to failures, which are system events.

2.3.22 Design Guidelines

Design can have a major effect on the amount of project resources expended during maintenance and the number of residual errors that occur during this process. A good quality system should provide design guidelines or checklist for reviewing the design at the higher level. A quality system, for example, might state that design containing multi-functional modules is to be discouraged (Design Principle; High COHESION). An auditor or a quality consultant should ask for these guidelines (if they exist in the organization visited) and read them thoroughly.

2.3.23 Acceptance Test Failures

Ask the testers in the organization what happens when an acceptance test fails. An undesirable answer is: we rectify the error, repeat the test and carry on. A desirable answer is: we rectify the error, repeat the test and the re-execute those tests, which exercise modules that had to be changed for rectifying the error(s). There is a high probability that a change necessitated by the discovery of an error will have caused further errors, which can only be discovered by re-running previous tests. The procedure for acceptance testing should ensure that this is a part of every acceptance test.

2.3.24 Project De-Briefings

Project de-briefings are a useful source of information used for improving the quality system. You should closely question the organization about the existence of standards and procedures governing the conduct of a de-briefing meeting and the associated documentation that is produced.

2.4 The Need for the SQA Group in an Organization

In an organization the Quality Policy, Quality Manual and Quality Management System form the 3-tier layer as shown in Figure 1. It should be noted that the terms SQA, depending on the context, might mean either the Software Quality Assurance as a function or Software Quality Analyst as a person. This should be kept in mind while reading the next few sections.

2.4.1 Why Does an Organization Need a Software Quality Assurance Function?

This question is asked very often! People find it difficult to be objective about auditors. After all, people feel that if they are doing their jobs carefully, why do they need any "third party" telling them if they are doing it in a right fashion. It is a natural human psychology to feel this way. The need for an "outside" or "third party" intervention, however, is a matter of perspective and a sign of organizational maturity. All of us would agree that while selecting a parachute or for that matter even a car, it is better to buy one that has been through a "proper" (i.e. strict) inspection. However, this is not how people and most organizations think when it comes to Software Quality, which is so vital for business survival as discussed in Chapter 1. The issues with software are not whether checks are needed (they are indeed needed) but who performs these checks and how (i.e. with proper procedures in place rather in a new ad-hoc fashion every time).

How large the SQA function is and how it is organized really depends on the size of the organization where it is employed. In very small organizations, it may not be practical/feasible to have a full time dedicated Software Quality Analysts. Also it may not be feasible to assign an SQA per project.

As the size of project staff grows, however, managers become involved with other duties like client liaison, taking care of statutory and regulatory requirements, reporting to senior management etc. They, then quickly lose touch with the day-to-day technical work. This is when they need to do one of the following:

- Find some way to handle their other workload so they can monitor more closely the people issues
- Hire someone to do the audit work
- Motivate the people to monitor each other

Of the alternatives indicated above, the third one is economically and technically most viable and so also from the morale point of view for the project team. Unfortunately, it does not work out practically; as the size of organization as well as the size of software projects within the organization grows beyond a few dozen people, these home grown "buddy systems" break down. This is when management needs to seriously think about deploying a Software Quality Assurance Function.

Software Quality Analyst role can be organized in one of the following ways; each project has a full time embedded SQA—i.e. a billable team member performing this role or have a member of the SEPG (*Software Process Engineering Group*) visit the project once in a while (frequency as pre-determined and documented) to check on adherence to processes or procedures defined in the Quality Management System. A typical thinking seen in Indian scenario is to assign the SQA role to developers who do not seem to be doing well in that job or assigning it to young and not-so-experienced members of the project team! This is only unfortunate for the software industry as a whole. It is time we realize that SQA (Software Quality Analyst) is a very responsible role with independent reporting to and close interaction with the senior management of the organization. As such this role should be respected.

2.4.2 The Role SQA Function: Roles and Duties

Although, we may often hear a cliché "Quality is Everyone's Responsibility!", in practice, the people responsible for the software projects' execution (the practitioners) are the only ones who can influence the quality culture in the organization along with the support from top management. The role of the SQA (Software Quality Assurance) function is to monitor the way the practitioners fulfill their responsibilities

towards quality commitment of the organization. The SQA function is the "Eyes and Ears of Management". In this context, a few common pitfalls are often noticed:

- You cannot assume that the SQAs (the Software Quality Analysts) themselves can do anything about quality
- The mere existence of an SQA function alone does not ensure that the standards and procedures are followed
- SQA will not be effective unless the top management shows its commitment through able leadership and periodically demonstrates its support for the SQA function by following its recommendations
- SQA program in an organization will not work effectively unless line management makes it clear that SQA try to resolve their issues with the project management before taking the route of escalation.

Thus, all that the SQA function can do is to alert management whenever deviations from established standards and procedures and practices occur. This is the primary responsibility of the Software Quality Assurance function. SQA can be effective when they report through an independent management chain. It is essential that the SQA function is staffed with qualified, competent and motivated individuals who are very committed to their job of quality assurance. It is difficult to get such people to work. For the SQA function to really perform like the "eyes and ears of management" performing objectively, the function (SQA) and quality professionals working in the team (the Software Quality Analysts) should be respected and be given the following responsibilities:

- Review all development and quality plans for completeness. It is possible to achieve this to some extent by deploying checklists designed with the help of the SEPG (Software Engineering Process Group) in an organization.
- Participate as inspection moderators (More on this will be discussed in the chapter on INSPECTIONS AND REVIEWS) while reviewing work-products of software projects (Design documents, code, user documentation etc.)
- Review of all Test Plans for adherence to standards. (An organization QMS could be mapped to international standards like ISO and CMM (*Capability Maturity Model*—this is discussed in greater details in a separate chapter devoted to process improvement frameworks)
- Review a significant sample of all test results to determine adherence to plans. (More on this in the chapter on TESTING)
- Periodically audit Software Configuration Plan (SCM) and configuration audit records to determine adherence to standards

(There is a separate chapter for detailed discussion on Software Configuration Management)

- Participate in all internal audits, co-ordinate external audits and register non-conformances (if any), participate in phase-end reviews of the projects, any other special project reviews as required.

For the SQA function to really fulfill this role in its fullest sense, it is the responsibility of the senior management to give adequate authority to this function. For example, declaring that a product cannot be shipped to customer unless and until the SQA has given a green signal by conducting the final release check (also known as Product Certification). Table 1 below provides an example of items that should be under SQA review.

Table 1: Items Under SQA Review

Item No	What the SQA does
1	Ensuring that a <i>requirements traceability matrix</i> or similar tool is used to show that the product specification cover the requirements
2	Verifying that an <i>implementation traceability matrix</i> or similar tool is used to show that the product specifications are implemented in the design.
3	Reviewing <i>project documentation</i> to verify that they are produced and maintained according to standards
4	Checking appropriate samples of <i>development records</i> (Project Plans, design review records, code review records, Test execution reports etc) to ensure that they are properly maintained and adequately represent the software design
5	Periodically verifying that <i>Software Configuration Management</i> (SCM) maintains proper baseline control as well as full records of change requests received for Requirements, Design, Code, Test and Documentation
6	Certifying that all the <i>sub-contractors' SQA functions</i> are adequately monitoring the performance of their organizations
7	Reviewing all <i>Plans</i> (Main as well as auxiliary) to ensure they include the required contents
8	Selectively <i>monitoring the performance</i> of the development, documentation, and test activities to ensure consistency with the approved plans and standards
9	Verifying that all specified <i>tests and peer reviews</i> are properly conducted, that the required results and data are recorded, and that suitable <i>follow-up actions</i> are performed and that <i>issues</i> are tracked to closure.
10	Auditing the <i>Change Control Board procedures</i> to verify that they are effectively implemented and planned
11	Selectively reviewing the resulting <i>design, code, and documentation</i> to ensure that they adhere to standards

2.4.3 The Goals of Software Quality Assurance Function

Goals of the SQA function can be broadly stated as:

- Improving software quality by appropriately monitoring both the software (Product Quality issues) and the processes used to produce it
- Ensuring full compliance with the established standards (based on frameworks and model like ISO, IEEE or CMM etc) and procedures for the software and software processes

- Ensuring that any inadequacies in the product, the processes, or the standards are brought to management's attention so that the in-adequacies are fixed on time.

Since the SQA function has such a entwined role with software development staff, at times it becomes tempting for the SQA staff members to get into "advisory" capacity and pass judgements that may not be called for. Although a technical/development background would be helpful, person playing the SQA role is not expected to play the "expert" role even when he or she may have had adequate development background. Nor should a person walk into an SQA function by looking upon it as place for his or her unfulfilled "development dreams" i.e. the mind-set: "now that I could not be a developer or be successful as developer, I'll show them how to do their job!" The skill of an SQA role lies in performing this entwined role in an objective and at times, in a remote fashion. A point to be noted and understood by both SQA and Management is that the SQA group or department is not responsible for "producing" quality products or for preparing projects' quality plans; this is the responsibility of the software project team members. SQA is a support function responsible for auditing the quality actions of the line organization and for altering management to any deviations and waivers. It is essential that a person performing SQA function keeps this focus all the time to be effective, SQA needs to work closely with development. They need to understand the project plans, verify the execution of these plans, and monitor the performance of the individual project tasks.

2.4.4 Functions of SQA

The basic organizational framework conducive to Software Quality Assurance Function is:

- **Quality Assurance Practices:** Adequate and state-of-the-art development tools, techniques, methods and standards should be defined and available for use. They should also be under periodic review.
- **Software Project Planning Evaluation:** Quality practices should be planned well in advance so that they can be implemented.
- **Requirements Evaluation:** You need high quality requirements to develop high quality products. Therefore, the key is reviewing the requirements right from the beginning to see if they conformance to the established quality standards. (Section 2.3.14 has already stressed the importance of this)
- **Evaluation of the design Process:** This was discussed in section 1.3.3. There should be appropriate mechanisms to ensure that the organisation follows the planned methodologies, that it implements the requirements, and that the quality of the design itself is adequately reviewed.

- **Evaluation of coding practices:** Appropriate coding standards/practices and guidelines must be established and used
- **Evaluating the software integration and test process:** A few remarks about this were made in sections 2.3.15 and 2.3.17. Testing should be performed (where possible) by an independent group that is both motivated and capable of finding defects. Test planning should begin early and the quality of the testing itself should be reviewed.
- **In-process evaluation of the management and project control process:** Sections 2.3.2, 2.3.8, 2.3.12, 2.3.13 and 2.3.18 had touched upon this. By ensuring that the management processes are working, SQA helps to see to it that the entire organization is focused on producing quality results.
- **Tailoring of Quality Assurance Procedures:** Guidelines must exist to tailor the QMS processes if required. The SQA plan should be tailored to the unique needs of each project and at times even to the needs of their customers. For example, in a situation where, adherence to customer's QA procedures is a part of contractual requirement, SQA should help do the mapping between the supplier organization's QMS and customer's QMS to avoid the need of duplication that may arise out of following both the sets of processes.

2.4.5 Who Should SQA Function Report To

Now that we have discussed the importance of the SQA function, the next logical question to discuss who should this function report to. Many differing practices will be found in organizations; some wrong, some right. Let's discuss here how the SQA reporting should be.

SQA function can be under a lot of flak from time to time both from the management as well as from the project staff. SQA role is tricky and it is easy for an SQA function to become ineffective. It is not a popular function in an organization since the job involves constantly telling people what is wrong! Without some motivating force and self drive, SQA people can degenerate to counting defects and arguing over petty and not so important matters (i.e. spelling mistakes and font selection in a project plan, for example!)

The one simple rule on SQA reporting is that it should not be under the software development manager. Projects face tight schedules and line managers are not likely to "listen" sympathetically to reports of inadequate test-plans, missing quality goals, defects found before release etc. Given this, SQA should report to a high-enough management level so that there is some chance of influencing project priorities and obtaining the resources and time to fix the key problems.

Reporting level is a matter of trade-off between, organization size, available choices etc. There are pros and cons with each option. With lower-level reporting, working relationships with development groups/